PostgreSQL SSL/TLS Configuration

Certificates (.crt / .key), Server Setup, Client Connections and Client-Certificate Authentication

Item	Value used in this guide
PostgreSQL	16 (applies to 12 and later, including 18)
Database server name	db1.example.com (IP 10.0.0.10)
Certificate directory	/etc/postgresql/ssl
Working directory for key generation	~/pg-certs (a secure admin machine is best)
Certificates created	ca.crt, ca.key, server.crt, server.key, client.crt, client.key
OS	Ubuntu/Debian (RHEL/Rocky differences noted)

Important: Replace the host names, IP addresses, organisation fields and paths with your own. Never share or commit any **.key** file; only .crt files are public.

Contents

- 1. How SSL/TLS works in PostgreSQL
- 2. Prerequisites
- 3. Create the certificates (self-signed quick lab method and private CA method)
- 4. Install the files on the server
- 5. Configure postgresql.conf
- 6. Configure pg_hba.conf
- 7. Reload PostgreSQL and verify the server side
- 8. Configure clients (sslmode, psql, JDBC, other drivers)
- 9. Client certificate authentication (mutual TLS)
- 10. Replication, pg_basebackup, PgBouncer and other tools
- 11. Enforce SSL and harden the setup
- 12. Verify and monitor
- 13. Renew, rotate and revoke certificates; Let's Encrypt option
- 14. Troubleshooting, cheat sheet and checklist

1. How SSL/TLS Works in PostgreSQL

PostgreSQL supports TLS (commonly called SSL) to encrypt the connection between clients and the server. This protects passwords, queries and results from being read or altered on the network. PostgreSQL uses OpenSSL and negotiates TLS 1.2 or 1.3 on the **same port** (5432); there is no separate SSL port.

Three levels of protection

Level	What you get	Needs
1. Encryption only	Traffic is encrypted, but the client cannot be sure it is talking to the real server (man-in-the-middle possible)	Server certificate and key; client sslmode=require
2. Server authentication	Level 1 plus the client verifies the server's certificate and host name	CA certificate on clients; sslmode=verify-full (recommended minimum for production)
3. Mutual TLS	Level 2 plus the server verifies a certificate presented by each client (can replace or add to passwords)	Client certificates; ssl_ca_file on server; pg_hba.conf clientcert or cert

Files involved

File	Type	Where it lives	Secret?
ca.key	CA private key	Offline / secure admin machine only	**Yes, most sensitive**
ca.crt	CA certificate (root)	Server (ssl_ca_file) and every client (sslrootcert)	No
server.key	Server private key	Database server only, mode 0600	**Yes**
server.csr	Certificate signing request	Temporary; can be deleted after signing	No
server.crt	Server certificate signed by the CA	Database server (ssl_cert_file)	No
client.key	Client private key	That client only, mode 0600	**Yes**
client.crt	Client certificate signed by the CA	That client (sslcert)	No

Remember: A **.crt** (certificate) is the public part and can be copied freely. A **.key** is the private part and must stay protected. The certificate and key must be a matching pair.

2. Prerequisites

Step 1 - Check OpenSSL and PostgreSQL SSL support

```
openssl version
pg_config --configure | grep -o "with-ssl=[a-z]*\|with-openssl"       # SSL compiled in
sudo -u postgres psql -c "SHOW ssl;"
sudo -u postgres psql -c "SHOW ssl_cert_file;" -c "SHOW ssl_key_file;" -c "SHOW data_directory;"
```

Packages from Debian/Ubuntu and PGDG are built with OpenSSL. If `openssl` is missing: `sudo apt install openssl` or `sudo dnf install openssl`.

Note: On Debian/Ubuntu, new clusters often already have `ssl = on` using a self-generated "snakeoil" certificate. That encrypts traffic but cannot be verified by clients. Replace it with your own certificate as shown in this guide.

Step 2 - Decide the names clients will use

With `sslmode=verify-full` the host name the client connects to must appear in the certificate's **Subject Alternative Name (SAN)**. List every DNS name and IP address clients will use (for example db1.example.com, a VIP, a load balancer name, 10.0.0.10). Decide this before generating the certificate.

Step 3 - Plan certificate lifetimes

- CA certificate: 5 to 10 years.
- Server and client certificates: 1 year (or less); set a renewal reminder (Section 13).

3. Create the Certificates

Choose one method. **Method B (private CA) is recommended** because it lets clients verify the server and lets you issue client certificates. If you have a company or public CA, you can instead request `server.crt` from it and skip to Section 4.

Method A - Quick self-signed certificate (lab / encryption only)

```
openssl req -new -x509 -days 365 -nodes -text \
  -out server.crt -keyout server.key \
  -subj "/CN=db1.example.com" \
  -addext "subjectAltName=DNS:db1.example.com,IP:10.0.0.10"
chmod 600 server.key
```

The certificate signs itself. For clients to verify it, copy `server.crt` to them and use it as their `sslrootcert`. This does not scale to many servers or client certificates, so use Method B for real environments.

Method B - Your own Certificate Authority (recommended)

B1. Create a working directory

```
mkdir -p ~/pg-certs && cd ~/pg-certs
umask 077            # new files readable only by you
```

B2. Create the CA key and CA certificate (ca.key, ca.crt)

```
# CA private key, protected by a passphrase (you will be prompted; store it in a password manager)
openssl genrsa -aes256 -out ca.key 4096

# Self-signed CA certificate valid for 10 years
openssl req -x509 -new -key ca.key -sha256 -days 3650 \
  -subj "/C=IN/ST=Telangana/O=MyOrg/CN=MyOrg PostgreSQL CA" \
  -addext "basicConstraints=critical,CA:TRUE" \
  -addext "keyUsage=critical,keyCertSign,cRLSign" \
  -out ca.crt
```

For a lab you may omit `-aes256` (no passphrase), but protect ca.key either way. Keep ca.key off the database servers.

B3. Create the server key (server.key)

```
openssl genrsa -out server.key 2048
```

B4. Create the server certificate request configuration

Create the file `server.cnf` with the names clients will use:

```
cat > server.cnf << 'EOF'
[req]
default_md         = sha256
prompt             = no
distinguished_name = dn
req_extensions     = v3_req

[dn]
C  = IN
ST = Telangana
O  = MyOrg
CN = db1.example.com

[v3_req]
basicConstraints = CA:FALSE
keyUsage         = digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth
subjectAltName   = @alt_names

[alt_names]
DNS.1 = db1.example.com
DNS.2 = localhost
IP.1  = 10.0.0.10
IP.2  = 127.0.0.1
EOF
```

B5. Create the CSR and sign it with the CA (server.csr, server.crt)

```
openssl req -new -key server.key -out server.csr -config server.cnf

openssl x509 -req -in server.csr \
  -CA ca.crt -CAkey ca.key -CAcreateserial \
  -out server.crt -days 365 -sha256 \
  -extfile server.cnf -extensions v3_req
```

B6. Verify what you created

```
# 1. Certificate chains to the CA
openssl verify -CAfile ca.crt server.crt                # expect: server.crt: OK

# 2. Names and validity dates
openssl x509 -in server.crt -noout -subject -issuer -dates
openssl x509 -in server.crt -noout -ext subjectAltName

# 3. Certificate and key are a matching pair (the two hashes must be identical)
openssl x509 -in server.crt -noout -pubkey | openssl sha256
openssl pkey -in server.key -pubout        | openssl sha256
```

B7. Optional: create client certificates (client.key, client.crt)

Only needed for mutual TLS (Section 9). The certificate's **CN should equal the PostgreSQL user name** (or be mapped in pg_ident.conf). Repeat for each user or application.

```
openssl genrsa -out client.key 2048
openssl req -new -key client.key -out client.csr -subj "/C=IN/O=MyOrg/CN=appuser"

cat > client_ext.cnf << 'EOF'
basicConstraints = CA:FALSE
keyUsage         = digitalSignature
extendedKeyUsage = clientAuth
EOF

openssl x509 -req -in client.csr \
  -CA ca.crt -CAkey ca.key -CAcreateserial \
  -out client.crt -days 365 -sha256 -extfile client_ext.cnf

openssl verify -CAfile ca.crt client.crt                # expect: client.crt: OK
```

Result of this section

```
~/pg-certs/
  ca.key      (secret, keep offline)       ca.crt      (public, goes to server and all clients)
  server.key  (secret, goes to DB server)  server.crt  (public, goes to DB server)
  client.key  (secret, goes to client)     client.crt  (public, goes to client)
  *.csr, *.cnf, ca.srl                     (working files; keep or delete)
```

4. Install the Files on the Server

Step 1 - Copy files to the database server

```
# from the admin machine
scp server.crt server.key ca.crt  admin@db1.example.com:/tmp/
```

Step 2 - Place them and set ownership and permissions

```
sudo mkdir -p /etc/postgresql/ssl
sudo mv /tmp/server.crt /tmp/server.key /tmp/ca.crt /etc/postgresql/ssl/

sudo chown postgres:postgres /etc/postgresql/ssl/server.crt /etc/postgresql/ssl/server.key \
                             /etc/postgresql/ssl/ca.crt
sudo chmod 700 /etc/postgresql/ssl
sudo chmod 600 /etc/postgresql/ssl/server.key        # REQUIRED: PostgreSQL refuses looser permissions
sudo chmod 644 /etc/postgresql/ssl/server.crt /etc/postgresql/ssl/ca.crt

ls -l /etc/postgresql/ssl
```

Permissions rule: PostgreSQL requires the key file to be 0600 and owned by the postgres user, or 0640 and owned by root (with the postgres group able to read it, as used by packaged system certificates). Anything more open prevents startup.

Note: RHEL family and SELinux: files under /etc/postgresql may have the wrong SELinux label. The simplest approach is to store the files in the data directory (/var/lib/pgsql/16/data) and use the relative names `server.crt`, `server.key`, `root.crt` in postgresql.conf, or run `sudo restorecon -Rv /etc/postgresql/ssl` after copying.

If the server certificate was signed by an **intermediate CA**, append the intermediate certificate(s) after the server certificate in `server.crt` (server certificate first, then intermediates). The root CA is not included.

```
cat server.crt intermediate.crt > server-chain.crt          # only when an intermediate CA is used
```

5. Configure postgresql.conf

Locate the file: `sudo -u postgres psql -c "SHOW config_file;"` (Debian: /etc/postgresql/16/main/postgresql.conf; RHEL: /var/lib/pgsql/16/data/postgresql.conf).

```
# ---- network (needed for remote clients) ----
listen_addresses = '*'                       # or the specific server IP, e.g. '10.0.0.10,127.0.0.1'
port = 5432

# ---- SSL / TLS ----
ssl = on
ssl_cert_file = '/etc/postgresql/ssl/server.crt'
ssl_key_file  = '/etc/postgresql/ssl/server.key'
ssl_ca_file   = '/etc/postgresql/ssl/ca.crt'     # required to verify CLIENT certificates (Section 9)
#ssl_crl_file = ''                                # certificate revocation list (Section 13)

ssl_min_protocol_version = 'TLSv1.2'             # TLSv1.3 if all clients support it
ssl_max_protocol_version = ''                    # empty = newest available
ssl_prefer_server_ciphers = on
ssl_ciphers = 'HIGH:!aNULL:!MD5'                 # applies to TLS 1.2 and below
#ssl_tls13_ciphers = ''                          # TLS 1.3 suites (available in PostgreSQL 18)
```

Parameter	Meaning
ssl	Turns TLS support on. Default off in upstream PostgreSQL.
ssl_cert_file	Server certificate (PEM). Default server.crt in the data directory.
ssl_key_file	Server private key (PEM). Default server.key in the data directory.
ssl_ca_file	CA certificate(s) used to verify client certificates. Needed for clientcert / cert authentication.
ssl_crl_file / ssl_crl_dir	Revocation list(s) for client certificates.
ssl_min_protocol_version	Oldest TLS version accepted. Default TLSv1.2; do not allow TLSv1 or TLSv1.1.
ssl_ciphers	OpenSSL cipher list for TLS 1.2 and lower. The default is acceptable; change only for compliance needs.
ssl_prefer_server_ciphers	Server's cipher preference wins when on.
ssl_passphrase_command	Command that supplies the passphrase if server.key is encrypted (Section 11).

6. Configure pg_hba.conf

Locate: `sudo -u postgres psql -c "SHOW hba_file;"`. The connection **type** controls whether SSL is required:

Type	Meaning
host	Matches both SSL and non-SSL TCP connections (does not enforce SSL).
hostssl	Matches only SSL (TLS) TCP connections.
hostnossl	Matches only non-SSL TCP connections.
local	Unix socket connections. These are not encrypted by SSL and do not need it.

Rules are evaluated top to bottom; the first match wins. Example that **requires SSL** for all remote connections:

```
# TYPE       DATABASE    USER        ADDRESS          METHOD
local        all         postgres                     peer
host         all         all         127.0.0.1/32     scram-sha-256       # local TCP, optional
hostssl      all         all         10.0.0.0/24      scram-sha-256       # SSL + password
hostssl      replication replicator  10.0.0.0/24      scram-sha-256       # streaming replication over S
hostnossl    all         all         0.0.0.0/0        reject              # refuse any non-SSL remote cl
```

Passwords are still verified. SSL only protects the connection, so keep strong authentication (`scram-sha-256`).

7. Reload and Verify the Server Side

Step 1 - Validate the HBA file before applying

```
sudo -u postgres psql -c "SELECT line_number, error FROM pg_hba_file_rules WHERE error IS NOT NULL;"
```

No rows means no syntax errors.

Step 2 - Apply the configuration

```
# Changes to ssl settings and pg_hba.conf only need a reload
sudo systemctl reload postgresql                  # Debian/Ubuntu
sudo systemctl reload postgresql-16               # RHEL family
# or: sudo -u postgres psql -c "SELECT pg_reload_conf();"

# If you changed listen_addresses, a restart is required:
sudo systemctl restart postgresql
```

Step 3 - Check the log and the settings

```
sudo tail -n 30 /var/log/postgresql/postgresql-16-main.log        # RHEL: log directory under the data d
sudo -u postgres psql -c "SHOW ssl;" -c "SHOW ssl_cert_file;" -c "SHOW ssl_min_protocol_version;"
```

The log must not contain errors like *could not load server certificate file* or *private key file has group or world access*.

Step 4 - Test the TLS handshake from another machine

```
openssl s_client -starttls postgres -connect db1.example.com:5432 -CAfile ca.crt -verify_hostname db1.ex
# Look for:  Verify return code: 0 (ok)   and   Protocol: TLSv1.3 (or TLSv1.2)
```

8. Configure Clients

8.1 The sslmode setting

sslmode	Encrypts?	Verifies server certificate?	Verifies host name?	Use
disable	No	No	No	Never for remote connections
allow	Only if server insists	No	No	Avoid
prefer (default)	Tries SSL, falls back to plain	No	No	Avoid: silently falls back to unencrypted
require	Yes	No (unless sslrootcert is set)	No	Encryption only; open to MITM
verify-ca	Yes	Yes	No	Better; trusts any host with a valid cert from the CA
verify-full	Yes	Yes	**Yes**	**Recommended for production**

8.2 Copy the CA certificate to the client

```
mkdir -p ~/.postgresql
cp ca.crt ~/.postgresql/root.crt          # default location libpq checks automatically
chmod 644 ~/.postgresql/root.crt
```

8.3 Connect with psql

```
psql "host=db1.example.com port=5432 dbname=appdb user=appuser sslmode=verify-full sslrootcert=/path/to/

# URI form
psql "postgresql://appuser@db1.example.com:5432/appdb?sslmode=verify-full&sslrootcert=/path/to/ca.crt"

# Using environment variables (good for scripts)
export PGSSLMODE=verify-full
export PGSSLROOTCERT=/path/to/ca.crt
psql -h db1.example.com -U appuser appdb
```

A successful connection prints a line like: *SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off)*.

8.4 Client parameter reference (libpq)

Parameter	Environment variable	Purpose
sslmode	PGSSLMODE	Security level (see table above)
sslrootcert	PGSSLROOTCERT	CA certificate used to verify the server. PostgreSQL 16+ accepts the value `system` to use the OS trust store (for publicly trusted certificates).
sslcert	PGSSLCERT	Client certificate (mutual TLS)
sslkey	PGSSLKEY	Client private key (must be 0600)
sslcrl	PGSSLCRL	Revocation list the client checks
sslpassword		Passphrase for an encrypted client key

8.5 JDBC (Java)

```
jdbc:postgresql://db1.example.com:5432/appdb?ssl=true&sslmode=verify-full&sslrootcert=/path/ca.crt

# Mutual TLS: the JDBC driver needs the client key in PKCS#8 DER format
openssl pkcs8 -topk8 -inform PEM -outform DER -in client.key -out client.pk8 -nocrypt
jdbc:postgresql://db1.example.com:5432/appdb?ssl=true&sslmode=verify-full\
&sslrootcert=/path/ca.crt&sslcert=/path/client.crt&sslkey=/path/client.pk8
```

8.6 Other drivers

Driver	Typical setting
Python psycopg / psycopg2	sslmode=verify-full, sslrootcert=/path/ca.crt (plus sslcert, sslkey for mutual TLS) in the connection string
SQLAlchemy	connect_args={"sslmode": "verify-full", "sslrootcert": "/path/ca.crt"}
node-postgres (pg)	ssl: { ca: fs.readFileSync('ca.crt'), rejectUnauthorized: true } (add cert and key for mutual TLS)
Go (pgx)	Connection string with sslmode=verify-full&sslrootcert=/path/ca.crt
.NET (Npgsql)	SSL Mode=VerifyFull;Root Certificate=/path/ca.crt (Client Certificate / Client Certificate Key for mutual TLS)
pgAdmin / DBeaver	In the connection's SSL tab choose verify-full and select the CA certificate (and client cert/key if used)

9. Client Certificate Authentication (Mutual TLS)

With mutual TLS the server also checks a certificate from each client. You can use it **in addition to** a password (two factors) or **instead of** a password.

Step 1 - Create client certificates (Section 3, B7) and give each client its own client.crt and client.key

The certificate's Common Name (CN) must equal the PostgreSQL user name, unless you use a name map (Step 4).

Step 2 - Make sure the server trusts the CA

```
# postgresql.conf
ssl_ca_file = '/etc/postgresql/ssl/ca.crt'
```

Step 3 - Choose the pg_hba.conf rule

```
# Option 1: password AND valid client certificate (CN must match the user)
hostssl  appdb  appuser  10.0.0.0/24  scram-sha-256  clientcert=verify-full

# Option 2: certificate only, no password (method "cert" implies verify-full)
hostssl  appdb  appuser  10.0.0.0/24  cert

# Option 3: certificate must be signed by the CA, CN not required to match the user
hostssl  appdb  appuser  10.0.0.0/24  scram-sha-256  clientcert=verify-ca
```

Step 4 - Optional: map certificate names to database users

Use when the CN (for example a host name) differs from the database user name.

```
# pg_hba.conf
hostssl  appdb  appuser  10.0.0.0/24  cert  map=certmap

# pg_ident.conf        MAPNAME   SYSTEM-USERNAME (certificate CN)   DATABASE-USERNAME
certmap                app-server-01                                appuser
certmap                app-server-02                                appuser
```

Step 5 - Install the files on the client and set permissions

```
mkdir -p ~/.postgresql
cp client.crt ~/.postgresql/postgresql.crt
cp client.key ~/.postgresql/postgresql.key
cp ca.crt     ~/.postgresql/root.crt
chmod 600 ~/.postgresql/postgresql.key              # REQUIRED; libpq refuses looser permissions
```

Step 6 - Reload and connect

```
sudo systemctl reload postgresql

psql "host=db1.example.com dbname=appdb user=appuser sslmode=verify-full \
      sslrootcert=/path/ca.crt sslcert=/path/client.crt sslkey=/path/client.key"
```

If the client certificate is missing or invalid, the connection fails with an error such as *connection requires a valid client certificate* or *certificate authentication failed for user*.

10. Replication, Backup Tools and Poolers

Streaming replication over SSL

```
# Primary pg_hba.conf
hostssl  replication  replicator  10.0.0.0/24  scram-sha-256

# Standby: base backup over SSL
pg_basebackup -h db1.example.com -U replicator -D /var/lib/postgresql/16/main -R -P \
  -d "sslmode=verify-full sslrootcert=/etc/postgresql/ssl/ca.crt"

# Standby primary_conninfo (postgresql.auto.conf is written by -R; check it contains SSL options)
primary_conninfo = 'host=db1.example.com user=replicator sslmode=verify-full sslrootcert=/etc/postgresql
```

The standby needs the CA certificate readable by the postgres user. Standbys also need their **own** server certificate if clients connect to them (add the standby's names to its certificate's SAN).

Logical replication / foreign data wrappers

Add `sslmode=verify-full sslrootcert=...` to the subscription `CONNECTION` string and to postgres_fdw server options.

PgBouncer

```
# pgbouncer.ini - clients to PgBouncer
client_tls_sslmode = require
client_tls_key_file = /etc/pgbouncer/server.key
client_tls_cert_file = /etc/pgbouncer/server.crt
client_tls_ca_file = /etc/pgbouncer/ca.crt

# PgBouncer to PostgreSQL
server_tls_sslmode = verify-full
server_tls_ca_file = /etc/pgbouncer/ca.crt
```

pgBackRest

pgBackRest normally connects to PostgreSQL over the local Unix socket, which does not use SSL. When repository and database are on different hosts, pgBackRest uses SSH or its own TLS server (separate certificates), which is configured independently of PostgreSQL's SSL.

11. Enforce SSL and Harden the Setup

- **Require SSL for all remote clients:** use `hostssl` rules and a final `hostnossl ... reject` rule (Section 6).
- **Use sslmode=verify-full** in every application and tool; avoid prefer and require.
- **Minimum protocol:** keep `ssl_min_protocol_version = 'TLSv1.2'` or higher.
- **Key protection:** server.key mode 0600, owned by postgres, never in version control or shared drives.
- **Keep the CA key offline** (admin workstation or HSM) and protect it with a passphrase.
- **Short lifetimes:** issue certificates for 1 year or less and automate renewal.
- **Strong authentication in addition to TLS:** scram-sha-256, and client certificates for sensitive services.
- **Firewall:** allow port 5432 only from needed networks.
- **Keep OpenSSL and PostgreSQL updated** for security fixes.

Optional: passphrase-protected server key

An encrypted server.key prevents use of a stolen key file but needs PostgreSQL to obtain the passphrase at startup.

```
# Encrypt an existing key
openssl pkey -in server.key -aes256 -out server.key.enc

# postgresql.conf
ssl_passphrase_command = '/usr/local/bin/get-pg-key-pass.sh'
ssl_passphrase_command_supports_reload = on         # allow reload without a restart
```

The script prints the passphrase to standard output and must itself be tightly protected (for example reading from a secrets manager or a root-only file). Without a passphrase command, an encrypted key can block automatic service start.

12. Verify and Monitor

Confirm a session is using SSL

```
-- Your own connection
SELECT ssl, version, cipher, bits, client_dn FROM pg_stat_ssl WHERE pid = pg_backend_pid();
-- In psql:  \conninfo

-- All client connections and whether they use SSL
SELECT a.usename, a.datname, a.client_addr, s.ssl, s.version, s.cipher, s.client_dn
FROM pg_stat_ssl s JOIN pg_stat_activity a USING (pid)
WHERE a.backend_type = 'client backend'
ORDER BY s.ssl, a.usename;

-- Count of unencrypted remote sessions (should be zero)
SELECT count(*) FROM pg_stat_ssl s JOIN pg_stat_activity a USING (pid)
WHERE a.backend_type='client backend' AND a.client_addr IS NOT NULL AND NOT s.ssl;
```

Check certificate expiry

```
openssl x509 -in /etc/postgresql/ssl/server.crt -noout -enddate
echo | openssl s_client -starttls postgres -connect db1.example.com:5432 2>/dev/null | openssl x509 -noo
```

Automated check (run daily from cron or your monitoring agent):

```
#!/bin/bash
# /usr/local/bin/check-pg-cert.sh : warn if the certificate expires within 30 days
CERT=/etc/postgresql/ssl/server.crt
if ! openssl x509 -checkend $((30*86400)) -noout -in "$CERT" >/dev/null; then
  echo "WARNING: PostgreSQL certificate $CERT expires within 30 days"; exit 1
fi
```

13. Renew, Rotate and Revoke

Renew a server certificate (about 30 days before expiry)

```
# On the admin machine: reuse the existing key, create a new CSR and sign it
openssl req -new -key server.key -out server.csr -config server.cnf
openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial \
  -out server.crt -days 365 -sha256 -extfile server.cnf -extensions v3_req
openssl verify -CAfile ca.crt server.crt

# Copy to the server, then
sudo cp server.crt /etc/postgresql/ssl/server.crt
sudo chown postgres:postgres /etc/postgresql/ssl/server.crt && sudo chmod 644 /etc/postgresql/ssl/server
sudo systemctl reload postgresql                # no restart or downtime needed
```

To rotate the private key as well, generate a new `server.key` first and replace both files. Clients need no change because they trust the CA, not the individual certificate.

Renew or replace the CA certificate

- Create the new CA and distribute a **bundle** containing both old and new CA certificates to servers (`ssl_ca_file`) and clients (`sslrootcert`) first.
- Then issue new server and client certificates from the new CA and deploy them.
- Finally remove the old CA from the bundle once all certificates are replaced.

```
cat ca-old.crt ca-new.crt > ca-bundle.crt
```

Revoke a compromised client certificate

Create a certificate revocation list (CRL) with your CA tooling (for example `openssl ca -gencrl`, which requires an OpenSSL CA database), copy it to the server and reference it:

```
# postgresql.conf
ssl_crl_file = '/etc/postgresql/ssl/ca.crl'
```

Reload PostgreSQL. Regularly regenerate and redeploy the CRL before it expires. For a small number of clients, issuing short-lived certificates and removing the user's access in pg_hba.conf is often simpler.

Alternative: publicly trusted certificate (Let's Encrypt)

If the server has a public DNS name, you can use a certificate from a public CA. Clients then trust it through the operating system store (`sslmode=verify-full sslrootcert=system` on PostgreSQL 16+ clients) without distributing your own CA. Use a deploy hook so renewals reach PostgreSQL:

```
# /etc/letsencrypt/renewal-hooks/deploy/postgresql.sh   (chmod +x)
#!/bin/bash
D=/etc/letsencrypt/live/db1.example.com
install -o postgres -g postgres -m 644 $D/fullchain.pem /etc/postgresql/ssl/server.crt
install -o postgres -g postgres -m 600 $D/privkey.pem   /etc/postgresql/ssl/server.key
systemctl reload postgresql
```

This works for hosts reachable for certificate validation. Client certificates for mutual TLS still need your own CA.

14. Troubleshooting, Cheat Sheet and Checklist

Error message	Cause and fix
private key file "server.key" has group or world access	Permissions too open. Run chmod 600 server.key and chown postgres:postgres (or 0640 root:postgres).
could not load server certificate file / key values mismatch	Certificate and key are not a pair, wrong path, or wrong format. Compare the public key hashes (Section 3, B6).
could not access private key file: Permission denied	The postgres user cannot read the file or a parent directory. Check ownership and directory permissions (and SELinux on RHEL).
SSL is not enabled on the server / server does not support SSL	ssl = off, not reloaded, or a certificate error at startup. Check the PostgreSQL log.
certificate verify failed	Client does not trust the issuer. Set sslrootcert to the correct ca.crt.
server certificate for "x" does not match host name "y"	The name used in the connection is not in the SAN. Connect with a listed name or re-issue the certificate with the right DNS/IP entries.
certificate has expired	Renew the certificate (Section 13). Check the server clock (NTP).
no pg_hba.conf entry for host ..., SSL off	Client connected without SSL while only hostssl rules match. Use sslmode=require or higher.
connection requires a valid client certificate	Server requires clientcert but the client sent none. Provide sslcert and sslkey.
certificate authentication failed for user	CN does not match the user and no pg_ident map; or certificate not signed by ssl_ca_file.
tlsv1 alert unknown ca (in server log)	The server does not trust the client certificate's issuer. Ensure ssl_ca_file contains the signing CA.
private key file "client.key" has group or world access (client)	chmod 600 on the client key.
Java: PKIX path building failed	JVM does not trust the CA. Use sslrootcert with the PostgreSQL JDBC driver, or import ca.crt into the Java truststore.
Server will not start after enabling an encrypted key	Passphrase unavailable. Configure ssl_passphrase_command or use an unencrypted key with strict permissions.

Command cheat sheet

Task	Command
Create CA key + cert	openssl genrsa -aes256 -out ca.key 4096 ; openssl req -x509 -new -key ca.key -sha256 -days 3650 -out ca.crt
Create server key	openssl genrsa -out server.key 2048
Create CSR	openssl req -new -key server.key -out server.csr -config server.cnf
Sign with CA	openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out server.crt -days 365 -sha256 -extfile server.cnf -extensions v3_req
Verify chain	openssl verify -CAfile ca.crt server.crt
Show SAN and dates	openssl x509 -in server.crt -noout -ext subjectAltName -dates
Reload PostgreSQL	sudo systemctl reload postgresql
Test TLS handshake	openssl s_client -starttls postgres -connect HOST:5432 -CAfile ca.crt
Check session SSL	SELECT ssl, version, cipher FROM pg_stat_ssl WHERE pid = pg_backend_pid();
Connect securely	psql "host=HOST dbname=DB user=USER sslmode=verify-full sslrootcert=ca.crt"

Final checklist

- Certificates created with correct SAN names; chain verified with openssl verify.
- server.key is 0600 and owned by postgres; ca.key is not on the database server.
- postgresql.conf: ssl = on, cert, key and CA paths set, minimum TLS 1.2.
- pg_hba.conf: hostssl rules in place and hostnossl reject rule last; pg_hba_file_rules shows no errors.
- Clients use sslmode=verify-full with the CA certificate (and client certificates where required).
- pg_stat_ssl shows no unencrypted remote sessions.
- Replication, poolers and tools configured for SSL.
- Expiry monitoring and a renewal procedure are in place.

References: postgresql.org/docs/current/ssl-tcp.html, postgresql.org/docs/current/libpq-ssl.html, postgresql.org/docs/current/auth-pg-hba-conf.html